Lab 7

Database Design using Entity-Relationship Diagrams (ERD)

CSE 4308

DATABASE MANAGEMENT SYSTEMS LAB

Prepared by:
Samnun Azfar, Jr. Lecturer, CSE
Samia Nawsheen, Jr. Lecturer, CSE
Academic Year: AY25-26

Contents

1 Introduction to Database Design **2**
1.1 Design Phases . . . 2
1.2 Common Design Pitfalls . . . 2

2 The Entity-Relationship (ER) Model **3**
2.1 Entity Sets . . . 3
2.2 Attributes . . . 3
2.2.1 1. Simple vs. Composite Attributes . . . 3
2.2.2 2. Single-valued vs. Multi-valued Attributes . . . 4
2.2.3 3. Derived Attributes . . . 4

3 Relationships **5**
3.1 Visual Representation . . . 5
3.2 Roles and Recursive Relationships . . . 5
3.2.1 Concept Definition . . . 5
3.2.2 The Necessity of Roles . . . 5
3.2.3 Implementation Note . . . 6
3.3 Attributes on Relationships . . . 6

4 Constraints and Cardinality **7**
4.1 Visual Notation (Arrow Styles) . . . 7
4.2 Types of Mapping Cardinalities . . . 7
4.3 Participation Constraints . . . 8

5 Advanced ER Features **9**
5.1 Weak Entity Sets . . . 9

6 Implementation: Solved Example (Complex ERD to Tables) **10**
6.1 The Scenario: University Academic System . . . 10
6.2 Step-by-Step Conversion . . . 10
6.2.1 Step 1: Strong Entities . . . 10
6.2.2 Step 2: Multi-valued Attributes . . . 11
6.2.3 Step 3: Weak Entities . . . 11
6.2.4 Step 4: Many-to-Many (M:N) Relationship . . . 11

1 Introduction to Database Design

The database design process is a structured approach used to transform real-world data requirements into a physical database implementation. It is crucial to ensure the final system is efficient, consistent, and capable of supporting user needs.

1.1 Design Phases

The design process is typically divided into three major phases:

1. **Initial Phase (Requirement Analysis):** This phase involves characterizing the data needs of prospective database users. The designer interacts with domain experts to generate a specification of user requirements. For instance, in a university context, this might involve identifying that the system needs to store data about students, instructors, and the courses they take.

2. **Conceptual-Design Phase:** Here, the designer translates the requirements into a conceptual schema. The **Entity-Relationship (ER) model** is the standard tool used for this purpose. The result is an ER Diagram (ERD), which is an abstract representation of the data, independent of the specific database software (like MySQL or Oracle) that will eventually be used.

3. **Implementation Phase:** This phase moves from the abstract to the concrete:

    - **Logical Design:** Maps the conceptual ER schema onto the implementation data model (usually the Relational Model, consisting of tables and constraints).
    - **Physical Design:** Handles storage details, such as file organization and indexing, to optimize performance.

1.2 Common Design Pitfalls

A robust design must avoid two specific errors:

- **Redundancy:** This occurs when the same piece of information is stored in multiple places. It is dangerous because updating the data in one location without updating it in others leads to *data inconsistency*.

- **Incompleteness:** A design is incomplete if it fails to model necessary aspects of the enterprise or introduces bad business logic, making it impossible to store required information.

2 The Entity-Relationship (ER) Model

The ER model represents data as a collection of entities and relationships.

2.1 Entity Sets

An **entity** is a distinct object in the real world that is distinguishable from all other objects. Examples include a specific person ("Einstein"), a company, or a course.

An **entity set** is a collection of entities of the same type that share the same properties (e.g., the set of all students).

Visual Representation:

- **Rectangles** represent entity sets.
- The name of the entity set is placed at the top or center.
- **Attributes** are listed inside the rectangle.
- The **Primary Key** (the attribute that uniquely identifies an entity) is underlined.

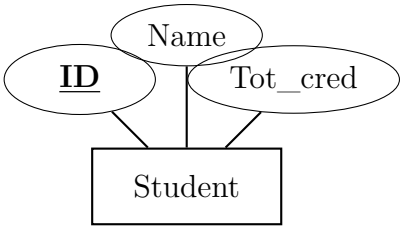


Figure 1: Example of an Entity Set. The attribute ID is the primary key.

2.2 Attributes

Attributes describe the properties of an entity. They are classified as follows:

2.2.1 1. Simple vs. Composite Attributes

- **Simple Attribute:** An attribute that cannot be divided further (e.g., `ID`, `Salary`).
- **Composite Attribute:** An attribute that has meaningful sub-parts. For example, a `Name` attribute might be composed of `First Name`, `Middle Initial`, and `Last Name`.
- **ERD Notation:** Composite attributes are connected to their parent attribute.

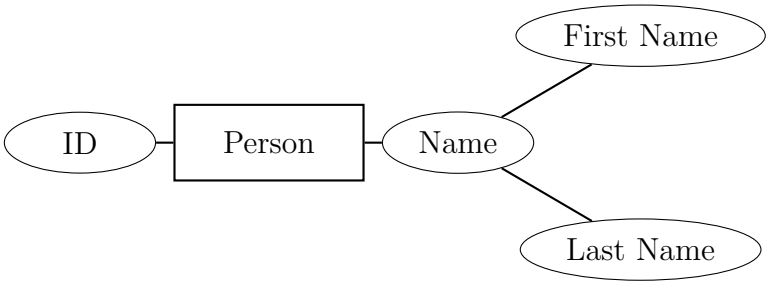


Figure 2: Simple Attribute (ID) vs. Composite Attribute (Name broken into parts).

2.2.2 2. Single-valued vs. Multi-valued Attributes

- **Single-valued:** The attribute holds a single value for a specific entity.
- **Multi-valued:** The attribute may hold multiple values (e.g., phone numbers). Multi-valued attribute allows for multiple entries for the same property.
- **ERD Notation:** Enclosed in a **double ellipse**.



Figure 3: Single-valued (ID) vs. Multi-valued Attribute (Phone), denoted by a double ellipse.

2.2.3 3. Derived Attributes

- The value is computed from other attributes (e.g., `Age` from `Date_of_Birth`).
- **ERD Notation:** Enclosed in a **dashed ellipse**.

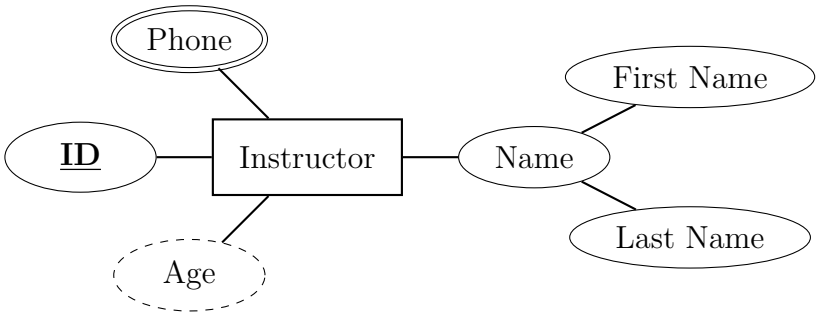


Figure 4: Entity with Composite (Name), Multi-valued (Phone), and Derived (Age) attributes.

3 Relationships

A **relationship** is an association between entities. For example, an `advisor` relationship links a `Student` entity with an `Instructor` entity.

3.1 Visual Representation

- **Diamonds** represent relationship sets.
- Solid lines connect the diamond to the participating entity sets.



Figure 5: A simple binary relationship connecting Instructor and Student.

3.2 Roles and Recursive Relationships

3.2.1 Concept Definition

In most relationships, we connect two *different* entity sets (e.g., a `Student` takes a `Course`). However, sometimes an entity set needs to relate to **itself**. This is called a **recursive relationship**.

Real-world Logic: Imagine a single table containing a list of all University Courses.

- *Course A* (Introduction to C) is a prerequisite for *Course B* (Data Structures).
- Both A and B belong to the same entity set: **Course**.
- We are linking a record in the `Course` entity to another record in the *same* `Course` entity.

3.2.2 The Necessity of Roles

Since both participants in the relationship are the same entity set, the meaning of the relationship can become ambiguous. To solve this, we use **Roles**.

- Roles are labels placed on the relationship lines to specify *how* the entity participates.
- In the prerequisite example, one instance of the course plays the role of the **Main Course** (the one having a requirement), and the other plays the role of the **Prerequisite** (the requirement itself).

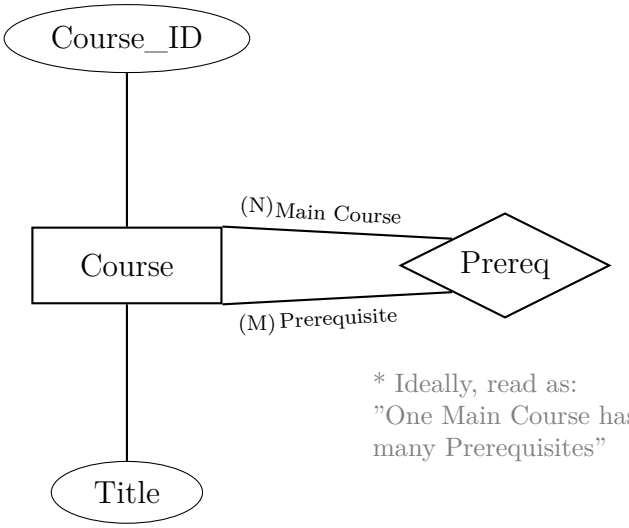


Figure 6: Recursive Relationship: The `Course` entity participates twice. Top line represents the course taking the requirement; bottom line represents the requirement itself.

3.2.3 Implementation Note

When mapping this to a database table (Implementation Phase), a recursive relationship typically results in a **Self-Join**. You would add a Foreign Key column (e.g., `prereq_id`) to the `Course` table that references the `course_id` of the same table.

3.3 Attributes on Relationships

Attributes can also be assigned to relationships. For instance, in an `Advisor` relationship, we might need to track the `date` the student was assigned to that instructor. This attribute is attached to the diamond.

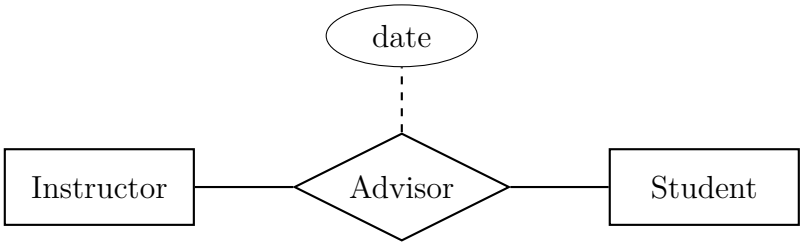


Figure 7: A relationship attribute (date) attached to the Advisor relationship.

4 Constraints and Cardinality

Mapping cardinality defines the number of entities to which another entity can be associated via a relationship set.

4.1 Visual Notation (Arrow Styles)

In standard ER diagrams, the style of the line tip indicates cardinality:

- **Directed Line (→):** Represents "One".
- **Undirected Line (—):** Represents "Many" (zero or more).

4.2 Types of Mapping Cardinalities

Type	Description	Diagram Style
One-to-One	An entity in A is associated with at most one in B, and vice versa.	→ on both sides
One-to-Many	An entity in A is associated with many in B. An entity in B is associated with at most one in A.	→ on A, — on B
Many-to-One	An entity in A is associated with at most one in B. An entity in B is associated with many in A.	— on A, → on B
Many-to-Many	Entities in both sets can be associated with any number of entities in the other set.	— on both sides

Table 1: Cardinality Mapping Styles.



Figure 8: **One-to-One**: A student has at most one advisor, and an instructor advises at most one student.



Figure 9: **One-to-Many**: An instructor can advise many students, but a student has only one advisor.



Figure 10: **Many-to-One**: An instructor is associated with at most one student, but a student may have many instructors.



Figure 11: **Many-to-Many**: An instructor advises many students, and a student has many advisors (e.g., joint supervision).

4.3 Participation Constraints

- **Total Participation:** Every entity in the set *must* participate in the relationship. This is indicated by a **double line**.
- **Partial Participation:** Some entities may not participate. This is indicated by a **single line**.



Figure 12: **Total Participation**: The double line connecting `Student` and `Advisor` indicates that every student must have an associated instructor.

5 Advanced ER Features

5.1 Weak Entity Sets

A **weak entity** is an entity that cannot be uniquely identified by its own attributes alone. It typically represents a dependency.

- It must be associated with a **strong entity** (owner entity).
- It uses a **discriminator** (partial key) combined with the strong entity's primary key for unique identification.

Diagram Notation:

- **Weak Entity Set:** Double Rectangle.
- **Identifying Relationship:** Double Diamond.
- **Discriminator Attribute:** Dashed Underline.

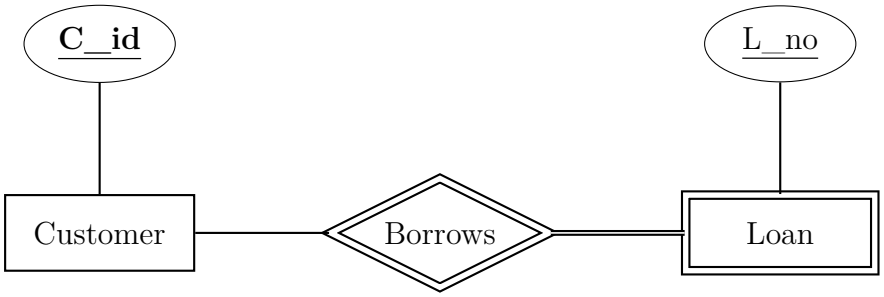


Figure 13: Weak Entity Example: `Loan` depends on `Customer`.

6 Implementation: Solved Example (Complex ERD to Tables)

Instead of memorizing abstract rules, let us walk through a complete, solved example of converting a complex ER Diagram into a Relational Schema (SQL Tables).

6.1 The Scenario: University Academic System

Consider the following ER Diagram representing a university system. It contains:

- **Strong Entities:** `Student` and `Course`.
- **Weak Entity:** `Exam` (Dependent on Course).
- **Multi-valued Attribute:** `Phone` on Student.
- **M:N Relationship:** `Enrolled` (Student takes Course) with a descriptive attribute `Grade`.

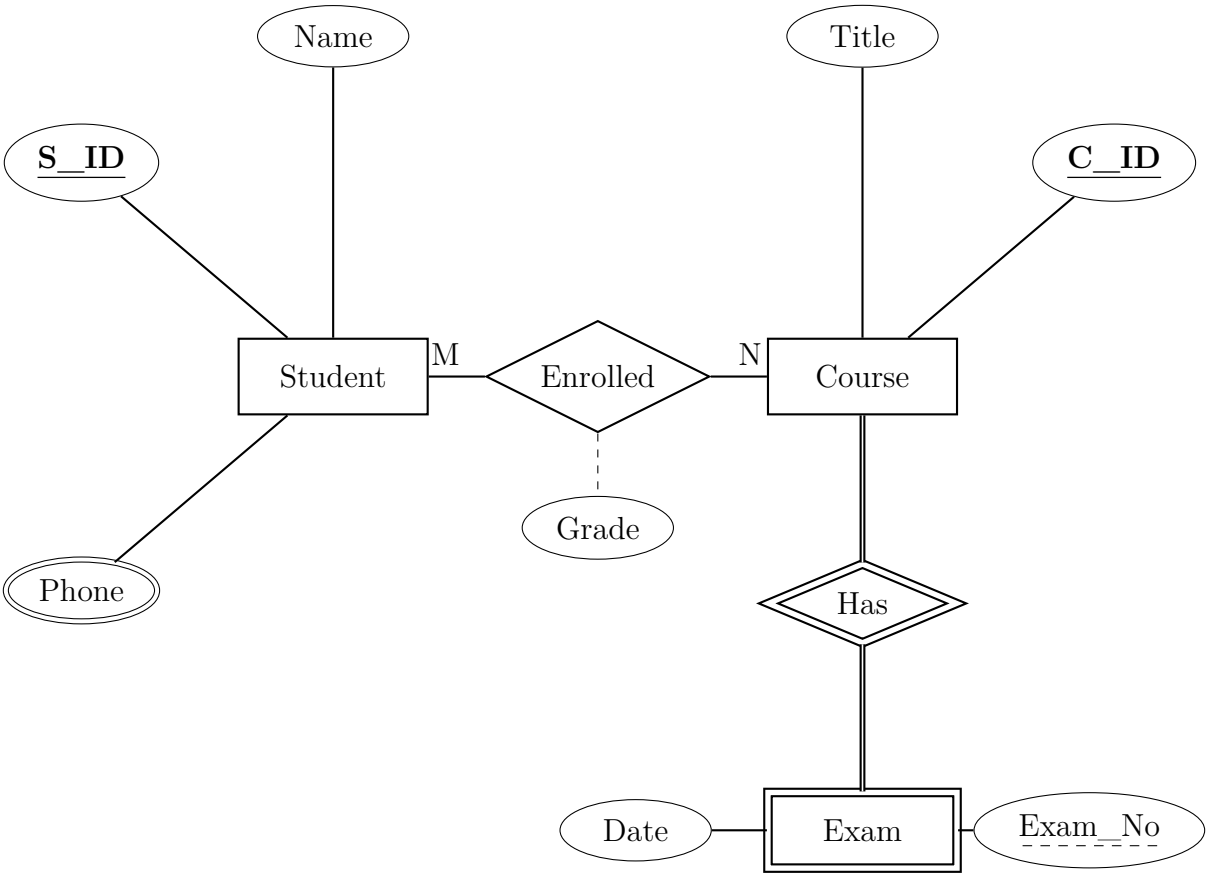


Figure 14: Complex ERD for conversion practice.

6.2 Step-by-Step Conversion

6.2.1 Step 1: Strong Entities

Create tables for independent entities. All simple attributes become columns.

```sql
CREATE TABLE Student (
    S_ID INT PRIMARY KEY,
    Name VARCHAR(100)
);

CREATE TABLE Course (
    C_ID VARCHAR(10) PRIMARY KEY,
    Title VARCHAR(100)
);
```

6.2.2 Step 2: Multi-valued Attributes

The `Phone` attribute on `Student` cannot be stored in the main `Student` table (violates 1NF). We create a separate table.

- **Columns:** Owner's Primary Key + The Attribute itself.
- **Composite Primary Key:** (S_ID, Phone).

```sql
CREATE TABLE Student_Phone (
    S_ID INT,
    Phone_Number VARCHAR(15),
    PRIMARY KEY (S_ID, Phone_Number),
    FOREIGN KEY (S_ID) REFERENCES Student(S_ID)
);
```

6.2.3 Step 3: Weak Entities

The `Exam` entity relies on `Course`. It has no unique ID of its own, only `Exam_No` (e.g., Exam 1, Exam 2).

- **Foreign Key:** Include `C_ID` from `Course`.
- **Composite Primary Key:** (C_ID, Exam_No).

```sql
CREATE TABLE Exam (
    C_ID VARCHAR(10),       -- From Strong Entity
    Exam_No INT,            -- Partial Key
    Exam_Date DATE,
    PRIMARY KEY (C_ID, Exam_No),
    FOREIGN KEY (C_ID) REFERENCES Course(C_ID)
        ON DELETE CASCADE   -- If Course is deleted, Exams are deleted
);
```

6.2.4 Step 4: Many-to-Many (M:N) Relationship

The `Enrolled` relationship links `Student` and `Course`. Since it is M:N, we cannot simply add a foreign key to either table. We need a **Junction Table**.

- **Columns:** FK from Student + FK from Course + Descriptive Attributes (Grade).
- **Composite Primary Key:** (S_ID, C_ID).

```sql
CREATE TABLE Enrolled (
    S_ID INT,
    C_ID VARCHAR(10),
    Grade CHAR(2),
    PRIMARY KEY (S_ID, C_ID),
    FOREIGN KEY (S_ID) REFERENCES Student(S_ID),
    FOREIGN KEY (C_ID) REFERENCES Course(C_ID)
);
```